DATA ENGINEERING MASTER PREP • EYE-CARE WARM PARCHMENT EDITION

Pillar 1: Storage Engines, Columnar Formats & Analytical SQL — Master Textbook

Row vs Columnar (Parquet), B-Tree vs LSM-Tree • Dimensional Modeling (SCDs) & Analytical SQL

INTERACTIVE DIRECTORY • CLICK ANY TOPIC TO JUMP

Table of Contents (Mục Lục)

7 Sections Indexed

Day 1: Row vs Columnar (Parquet), B-Tree vs LSM-Tree ..... p. 2

1.1 Storage Engine Internals: B+Tree (OLTP) vs LSM-Tree (Write-Heavy) (DE01) ..... p. 2

1.2 Apache Parquet Architecture: PAX Layout, Pushdowns & Encodings (DE02) ..... p. 3

1.3 Dremel / Parquet Striping: Repetition Level (R) & Definition Level (D) (DE03) ..... p. 4

Day 2: Dimensional Modeling (SCDs) & Analytical SQL ..... p. 4

2.1 Kimball Dimensional Modeling & Slowly Changing Dimensions (DE04) ..... p. 4

2.2 Analytical SQL Traps: Window Frames & Three-Valued NULL Logic (DE05 , DE06) ..... p. 4

> 💡 **TIP**
>
> **How to Use This Guide:** Every section below maps directly to your `DE01` – `DE06` Data Engineering questions. Use the **Contents** menu to jump to any sub-topic, or enable 📄 **Textbook: on** in 🤖 **Ask AI** so the AI tutor cites these exact derivations and tables.

Day 1: Row vs Columnar (Parquet), B-Tree vs LSM-Tree

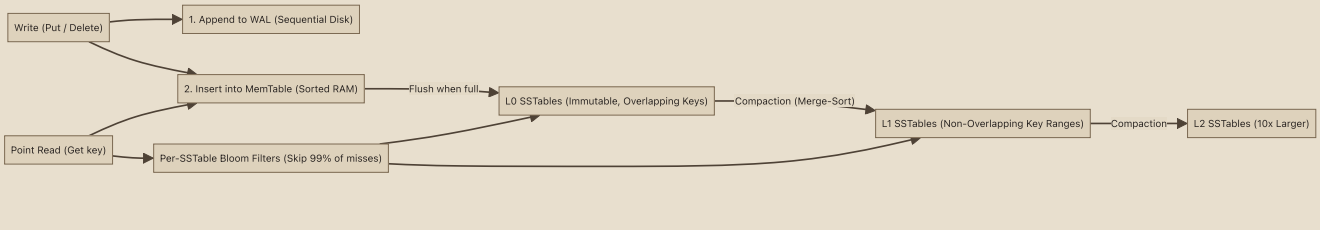
↑ Mục Lục

1.1 Storage Engine Internals: B+Tree (OLTP) vs LSM-Tree (Write-Heavy) (`DE01`)

💡 Intuition (Plain English First)

Every database storage engine optimizes a fundamental trade-off between **Read Amplification**, **Write Amplification**, and **Space Amplification** (the **RUM Conjecture**):

1. **B+Tree (PostgreSQL, MySQL InnoDB, Oracle):**
   - Stores data in fixed-size mutable disk pages (8–16 KB) organized as a balanced B-ary search tree where all rows live in the leaf pages.
   - **Strengths:** Point lookups and range scans touch only $\mathcal{O}(\log_B N)$ pages (2–4 disk seeks).
   - **Weakness:** Updating a single 100-byte row on a random key requires reading an entire 16 KB page, modifying it, and flushing the 16 KB page back to disk (**random I/O + high write amplification**).
2. **Log-Structured Merge-Tree (LSM-Tree — RocksDB, Apache Cassandra, HBase, ScyllaDB):**
   - **Never modifies disk files in place!** Every insert, update, or delete (`tombstone`) is appended sequentially to an on-disk **Write-Ahead Log (WAL)** and inserted into a sorted in-memory buffer (**MemTable**, typically a SkipList or B-Tree).
   - When the MemTable reaches capacity (e.g., 64 MB), it is flushed sequentially to disk as an **immutable Sorted String Table (`SSTable`)** at Level L_0.
   - Background **Compaction** threads periodically merge-sort overlapping SSTables from $L_k \rightarrow L_{k+1}$, discarding overwritten values and expired tombstones.



📐 Side-by-Side Comparison (`DE01`)

Metric / Property	B+Tree (`InnoDB`, `PostgreSQL`)	LSM-Tree (`RocksDB`, `Cassandra`, `HBase`)
Write I/O Pattern	**Random I/O** (in-place 8–16 KB page overwrites + page splits)	**Sequential I/O** (append-only WAL + immutable SSTable flushes)

Metric / Property	B+Tree (`InnoDB` , `PostgreSQL`)	LSM-Tree (`RocksDB` , `Cassandra` , `HBase`)
Point Read Latency	**Fast & predictable** ($\mathcal{O}(\log_B N)$, 1 leaf page)	**Higher tail latency** (must check MemTable $+L_0 + L_1 \ldots$; mitigated by **Bloom filters**)
Delete Mechanism	Removes entry directly from leaf page	Writes a **Tombstone** marker; space is reclaimed only during **Compaction**
Best Workload	Read-heavy / transactional **OLTP**	Write-intensive event ingestion, time-series, wide-column stores

1.2 Apache Parquet Architecture: PAX Layout, Pushdowns & Encodings (`DE02`)

💡 Intuition (Plain English First)

Why is **Apache Parquet** the universal storage format for modern Data Lakes and Lakehouses? Because it uses the **PAX (Partition Attributes Across)** hybrid layout:

1. **Horizontal Slicing into Row Groups (**$\sim$ 128 MB–1 GB**):** The table is first partitioned into large horizontal batches of rows (e.g., 10^6 rows per Row Group) so distributed workers (Spark executors) can process different Row Groups in parallel.
2. **Vertical Slicing into Column Chunks:** Inside each Row Group, all values of Column A are stored contiguously as a **Column Chunk**, followed by all values of Column B, and so on.
3. **Pages (**1 MB**):** Each Column Chunk is subdivided into **Dictionary Pages** and **Data Pages** with per-page `min` / `max` statistics.
4. **File Footer at the End of the File:** Contains the schema and every Row Group's Column Chunk byte offsets, `min` , `max` , and `null_count` .
   - *Why is the metadata in the **footer** instead of the header?* Because Parquet files are written in a single streaming pass! You don't know the byte offsets or `min` / `max` statistics of all Row Groups until the final row has been written!

📐 The Two I/O Pruning Mechanisms (`DE02`)

1. **Projection Pushdown (Column Pruning):** If a table has $C = 50$ columns and your query references $k = 3$ columns, the reader reads the footer first and issues byte-range reads for **only those** 3 **Column Chunks**, eliminating $\frac{50-3}{50} = 94\%$ of column I/O.
2. **Predicate Pushdown (Row Group / Page Skipping):** For a filter `WHERE event_date = '2026-11-01'` , the reader checks each Row Group's `[min, max]` in the footer:

$$\text{Skip Row Group } i \iff \text{'2026-11-01'} < \min_i(\text{event_date}) \ \vee \ \text{'2026-11-01'} > \max_i(\text{event_date})$$

> ⚠ **WARNING**
>
> **Exam Trap:** If `event_date` is unsorted (randomly distributed across all Row Groups), then **every** Row Group has min $\approx$ earliest date and max $\approx$ latest date, so `min` / `max` pruning skips 0 **Row Groups**! Always **sort** or **Z-Order** Parquet files by your high-selectivity filter columns.

1.3 Dremel / Parquet Striping: Repetition Level (R) & Definition Level (D) (`DE03`)

📐 Exact Rules for Nested Schemas (`DE03`)

Given a nested path from root to leaf `a.b.c.leaf`:

- **Maximum Repetition Level (R_{max}):** Equals the number of `repeated` fields along the path (`0` if there are no arrays/lists).
  - For each emitted value, $R \in [0, R_{max}]$ tells the reader **at which repeated ancestor depth the list repeated** compared to the previous value ($R = 0$ means a brand-new top-level record).
- **Maximum Definition Level (D_{max}):** Equals the number of `repeated` + `optional` fields along the path (`required` fields contribute `0` because they can never be missing!).
  - When a leaf value is non-null, $D = D_{max}$. When a value is `NULL`, $D < D_{max}$ tells the reader **how many optional/repeated ancestors were defined** before the `NULL` occurred.

Day 2: Dimensional Modeling (SCDs) & Analytical SQL

↑ Mục Lục

2.1 Kimball Dimensional Modeling & Slowly Changing Dimensions (`DE04`)

📐 Complete SCD Reference Table (`DE04`)

SCD Type	Mechanism	History Preserved?	Storage / Join Rule
Type 0	Retain original attribute value forever	Original only	Never updated (e.g., `signup_date`)
Type 1	**Overwrite** attribute in place	**No** (destroys history)	Historical facts retroactively switch to the new attribute
Type 2	**Add a new row** with a new **Surrogate Key** (`_sk`), `(valid_from, valid_to, is_current)`	**Yes (Unlimited)**	Fact table **must join on Surrogate Key** (`_sk`) (or natural key + timestamp between `valid_from` and `valid_to`)
Type 3	**Add a new column** (`prev_city`, `effective_date`)	**Only 1 prior state**	Cannot attribute historical facts across multiple changes

---

2.2 Analytical SQL Traps: Window Frames & Three-Valued `NULL` Logic (`DE05`, `DE06`)

1. The Window Frame Default Trap (`DE05`)

Whenever you write `<func>() OVER (PARTITION BY p ORDER BY o)` **without** an explicit `ROWS` or `RANGE` frame:

- SQL defaults to:

```
RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
```

- **Trap 1 (`LAST_VALUE`):** Because the window ends at `CURRENT ROW`, `LAST_VALUE(x) OVER (PARTITION BY dept ORDER BY dt)` returns the **current row's** `x`! Always write:

```sql
LAST_VALUE(salary) OVER (
  PARTITION BY dept ORDER BY hire_date
  ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
)
```

- **Trap 2 (`RANGE` vs `ROWS` on duplicate `ORDER BY` values):** `RANGE` treats all rows with equal `ORDER BY` values as a single peer group and sums them together on the first tied row. Use `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` when you want a strict row-by-row running sum.

2. The `NOT IN` + `NULL` Three-Valued Logic Trap (`DE06`)

In SQL, `x NOT IN (a, b, NULL)` is parsed as:

$$(x \neq a) \ \wedge \ (x \neq b) \ \wedge \ \underbrace{(x \neq \texttt{NULL})}_{\texttt{UNKNOWN}} \implies \texttt{UNKNOWN}$$

Because `WHERE` filters out both `FALSE` and `UNKNOWN`, **a single `NULL` inside a `NOT IN` subquery causes the outer query to return 0 rows**!

- **Safe Idioms:** Always use `WHERE NOT EXISTS (SELECT 1 FROM sub WHERE sub.id = main.id)` or `LEFT ANTI JOIN`.